

Module 7: Lesson 4

Advanced topics in Deep Learning for Finance: Custom Losses and Backtest Metrics



Outline

- ▶ Custom Loss functions and Backtest metrics
- ▶ Application to Bitcoin

Building custom losses in TensorFlow-Keras

Most of the time we use our models to develop investment strategies, whose performance metrics are defined in terms of risk-adjusted returns and other variables.

Typical loss functions that measure the predictive power of the models may not be appropriate targets or may not provide a general view of the quality of a predictive model.

In this lesson, we provide guidelines on how to build custom losses that allow for keeping track of the trading performance of the model while training and backtesting.

- The aim is that, after reading and analyzing the associated notebook, you can build loss functions and metrics that are tailored to the applications that you may encounter in the future.

General approach

Any function defined with the signature

```
loss_fn(y_true, y_pred)
```

can be passed to `compile()` as a loss function.

Both inputs are tensors with dimension (batch_size, number of outputs).

For instance, we can define a custom MSE loss function for a regression problem as follows and then pass it to our model.

```
def custom_mse(y_true, y_pred):  
    loss0 = tf.square(y_pred - y_true)  
    loss1 = tf.reduce_sum(loss0, axis=-1)  
    return loss1  
model.compile(loss=custom_mse, optimizer='adam')
```

At each optimization step, `fit()` will compute the average of “loss” across observations in the current batch.

Adding parameters

If our loss functions include some parameter, we need to define a **wrapper function** that includes the parameter of interest as an argument and that also nests the loss function of interest.

For instance, suppose that our model generates several outputs, and we want to give particular relevance to some of those outputs over others with a vector of weights.

```
def wrapper(w) # w is the vector of weights
    def custom_mse(y_true, y_pred):
        loss = tf.square(y_pred - y_true)
        loss_w = tf.math.multiply(loss, w) # apply weights
        loss_w_sum = tf.reduce_sum(loss_w, axis=-1)
        return loss_w_sum
    return custom_mse
```

We can pass the wrapper function to the compile method, after specifying the vector of weights.

```
weights = [0.1, ..., 0.025]
model.compile(loss=wrapper(weights), optimizer='adam')
```

Custom backtest statistics (1): Accuracy

We can apply the same logic to define backtest metrics that evaluate the performance of the trading strategy that arises from the trained model.

The backtest metrics that we define next are based on and tailored to the application of Lesson 4 focused on the prediction of Bitcoin returns.

For instance, we can compute an accuracy metric that measures the frequency with which the sign of the return prediction is the same as the actual prediction:

```
def custom_metric_acc(y_true , y_pred):  
    trues = tf.cast(tf.math.multiply(y_true , y_pred) > 0, tf.int32)  
    trues_sum = tf.reduce_sum(trues)  
    acc = trues / tf.size(y_pred)  
    return acc
```

Custom backtest statistics (2): R^2

We can also keep track of the model's R^2 over our backtest.

```
def custom_metric_r2(benchmark):  
    def r2(y_true, y_pred):  
        ss_pred = tf.math.reduce_sum(tf.square(y_true - y_pred))  
        ss_b = tf.math.reduce_sum(tf.square(y_true - benchmark))  
        return 1 - ss_pred / ss_b  
    return r2
```

We would obtain the typical out-of-sample R^2 if we set “benchmark” equal to the average return in the training sample and compute the function above in the test set.

Custom backtest statistics (3): Sharpe Ratios

Instead of focusing on forecasting power, we can directly focus on risk-weighted return performance of the strategy, through the Sharpe ratio.

```
def custom_metric_sr(b):  
    def sharpe_ratio(y_true, y_pred):  
        strat_ret = strategy_rets(y_true, y_pred, b)  
        ret_mean = tf.reduce_mean(strat_ret)  
        ret_std = tf.math.reduce_std(strat_ret)  
        sr = tf.math.divide(ret_mean, ret_std)  
        sr = tf.math.sqrt(252) * sr  
        return sr  
    return sharpe_ratio
```

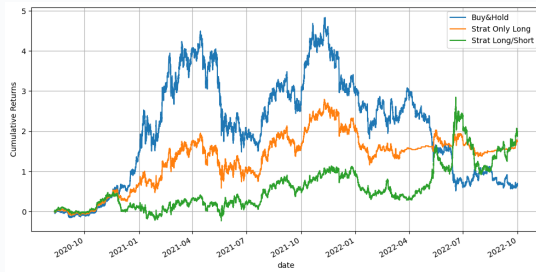
“strategy_rets” must be a function that determines (i) the position taken on the security given the predicted value and, possibly, some parameter(s) “b” and (ii) the return obtained given the position and the actual return.

Application

In the Jupyter Notebook for Lesson 4, we implement the construction of custom measures revisiting the Bitcoin return prediction problem that we already explored in Lesson 2.

We implement a Walk Forward procedure that keeps track of the backtesting metrics defined above.

- We also build other interesting metrics such as the Maximum Drawdown and Time under Water.



Summary of Lesson 4

In Lesson 4, we have:

- ▶ Built custom losses and backtest metrics in TensorFlow-Keras.
- ▶ Applied the computation of custom metrics in the evaluation of DL trading algorithms.

⇒ **References for this lesson:**

Anis, A. "How To Build Custom Loss Functions in Keras for Any Use Case." *convrg.io*.

López de Prado, M. "Advances in Financial Machine Learning: Lecture 6/10 (seminar slides)." *SSRN*. 6 October 2018.

TO DO NEXT: Please go to the Jupyter Notebook associated with Lesson 4.

This finalizes Module 7 and our journey in Deep Learning for Finance. Congrats!